

PEMROGRAMAN FASE 1

Nama : Adenia Azzahra
NIM : 26/577476/TK/65676
Program Studi : Teknologi Informasi
Kelompok : Onyx

Problem 1: The Last Astronaut

Pseudocode:

DEKLARASI

N, K : integer
astronaut : integer
jumlah : integer
index : integer
tereliminasi : integer
i : integer

DESKRIPSI

INPUT N
INPUT K

UNTUK $i \leftarrow 0$ SIMPAN N-1

Astronaut[i] $\leftarrow i + 1$

SELESAI

jumlah $\leftarrow N$

index $\leftarrow 0$

SELAMA jumlah > 1 LAKUKAN

index $\leftarrow (\text{index} + K - 1) \text{ MOD } \text{jumlah}$

tereliminasi $\leftarrow \text{astronaut}[\text{index}]$

TULISKAN tereliminasi

JIKA tereliminasi MOD 2 = 0 MAKA

K $\leftarrow K + 2$

JIKA TIDAK

K $\leftarrow K - 1$

JIKA K < 2 MAKA

K $\leftarrow 2$

AKHIR JIKA

AKHIR JIKA

UNTUK $i \leftarrow \text{index}$ SAMPAI jumlah - 2

astronaut[i] $\leftarrow \text{astronaut}[i + 1]$

SELESAI

```
jumlah ← jumlah – 1
```

```
JIKA index = jumlah MAKA
```

```
    index ← 0
```

```
AKHIR JIKA
```

```
SELESAI
```

Penjelasan Program:

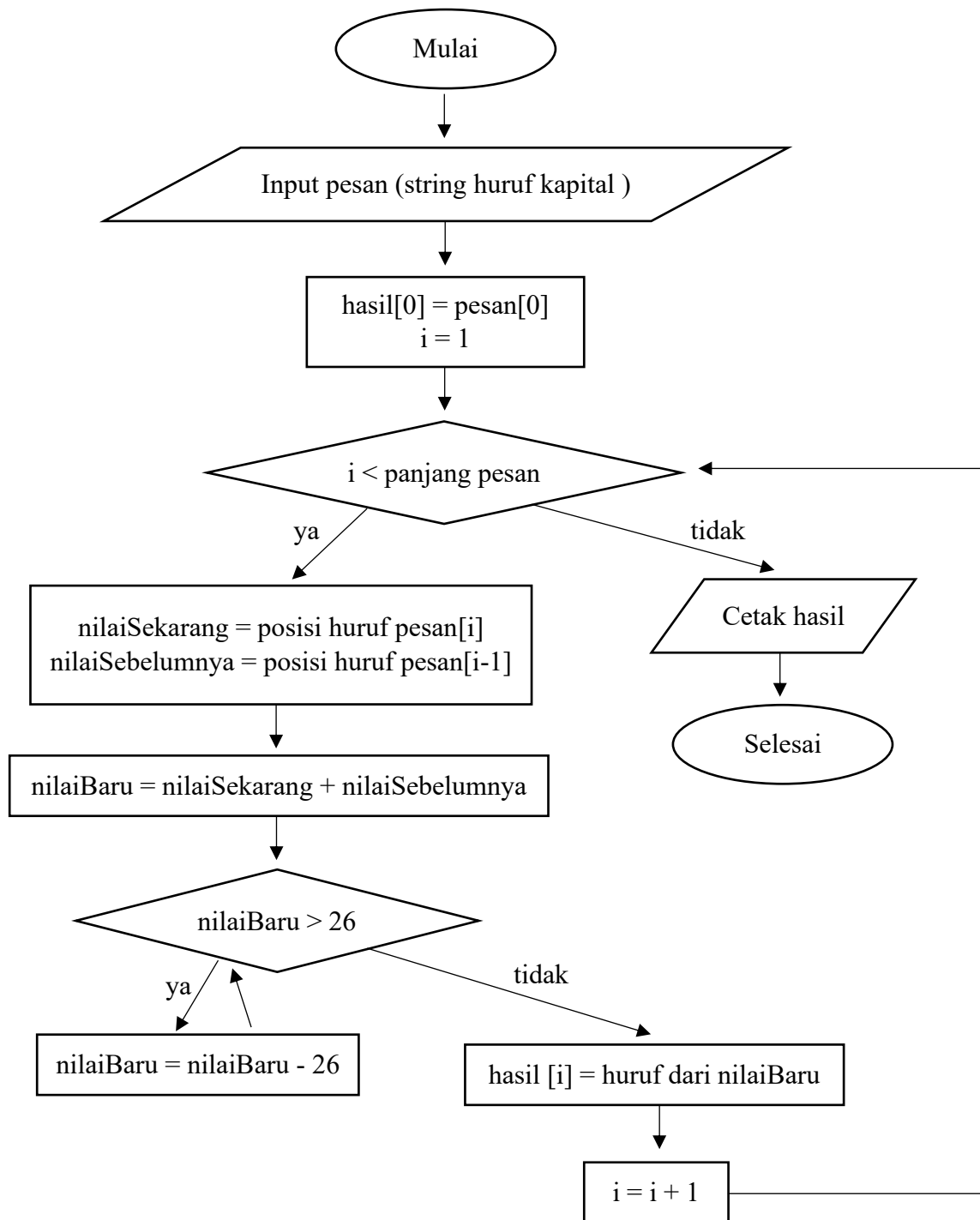
Program ini mempresentasikan lingkaran astronot menggunakan array `astronaut[]`, di mana array tersebut hanya berisi nomor-nomor astronot yang masih hidup, yang tersusun rapat mulai dari indeks 0 hingga jumlah 1. Dalam program ini, ketika astronot dieliminasi, elemen tersebut langsung dihapus secara fisik dari array dengan menggeser seluruh elemen setelahnya maju satu posisi, sehingga variabel jumlah yang menyatakan banyaknya astronot aktif benar-benar berkurang setiap kali terjadi eliminasi.

Untuk menentukan astronot mana yang harus dieliminasi pada setiap putaran, digunakan rumus $\text{index} \leftarrow (\text{index} + K - 1) \text{ MOD } \text{jumlah}$. Rumus ini akan menghitung posisi ke-K dari titik index saat ini tanpa perlu melakukan pengulangan hitung satu per satu. Karena posisi index dihitung sebagai urutan pertama, maka untuk mencapai hitungan ke-K diperlukan tambahan sebanyak K-1 langkah. Operasi modulo terhadap jumlah inilah yang membuat efek melingkar tercapai, karena begitu hasil penjumlahan melebihi index terakhir array, hasilnya otomatis akan kembali berputar ke index awal. Setelah posisi tersebut ditemukan, nilai pada `astronaut[index]` dicatat sebagai astronot yang tereliminasi, kemudian nilai K diperbarui mengikuti aturan soal, yakni bertambah 2 jika nomor yang dieliminasi genap, atau berkurang 1 jika ganjil, dengan batasan bawah K tidak boleh kurang dari 2.

Setelah astronot pada posisi index dihapus dan jumlah dikurangi satu, algoritma akan melakukan pengecekan tambahan, yaitu jika index sama dengan jumlah, maka index harus dikembalikan ke 0 agar perhitungan pada putaran berikutnya tetap merujuk ke posisi yang valid di dalam array. Seluruh proses menghitung index, mencatat eliminasi, memperbarui K, menggeser array, dan mengurangi jumlah akan diulang terus dalam satu struktur perulangan SELAMA (WHILE) hingga variabel jumlah bernilai 1. Ketika kondisi akhir tersebut tercapai, satu-satunya elemen yang tersisa di `astronaut[0]` merupakan astronot terakhir yang bertahan, sehingga nilai itulah yang dicetak sebagai hasil akhir dari algoritma.

Problem 2: Alien in The Middle

Flowchart:



Penjelasan:

Program ini adalah bentuk sandi substitusi berantai, di mana nilai geser suatu huruf ditentukan oleh huruf asli yang berada tepat sebelumnya. Setiap huruf dipetakan ke sebuah angka berdasarkan urutannya di alfabet menggunakan operasi sederhana $c - 'A' + 1$ pada representasi karakter ASCII. Huruf pertama pada pesan tidak mengalami perubahan sama sekali karena tidak memiliki huruf sebelumnya untuk dijadikan acuan pergeseran.

Untuk setiap huruf berikutnya, program mengambil nilai posisi huruf saat ini dan menjumlahkannya dengan nilai posisi huruf sebelumnya pada pesan asli. Jika program keliru menggunakan huruf yang sudah tersandi sebagai acuan, hasil akhirnya akan berbeda dari contoh pada soal (ALIENS menjadi AMUNSG). Hasil penjumlahan dua nilai posisi tersebut bisa melebihi 26 yang merupakan nilai maksimum untuk huruf Z, sehingga diperlukan mekanisme pembungkusan (*wrap-around*). Selama nilai masih lebih besar dari 26, nilai tersebut dikurangi 26 hingga berada kembali pada rentang 1-26.

Setelah nilai akhir berada pada rentang yang valid, nilai tersebut dikonversi kembali menjadi karakter huruf menggunakan operasi kebalikan dari pemetaan awal, yaitu $(\text{nilai} - 1) + \text{'A'}$. Proses ini diulang untuk seluruh huruf dalam pesan hingga terbentuk pesan tersandi yang lengkap. Karena hanya dipernolehkan memakai iostream tanpa fungsi pustaka bawaan, seluruh proses seperti menghitung panjang string, mengonversi huruf ke angka, mengonversi angka ke huruf, dan proses enkripsi utama masing-masing dibuat sebagai fungsi manual (*user-defined function*) yang saling dipanggil satu sama lain di dalam fungsi main.